

Formal Languages and Compilers (Linguaggi Formali e Compilatori)

prof. Luca Breveglieri (prof. S. Crespi Reghizzi, prof. A. Morzenti)

Written exam - 28 February 2013 - Part I: Theory

WARNING - STARTING FROM THE SUMMER SESSION (JUNE 2013), THE EXERCISES ON SYNTAX ANALYSIS MUST BE CARRIED OUT BY THE THEORY *ELR* / *ELL* - THE THEORY *LR* / *LL* WILL NOT BE ACCEPTED ANY LONGER - SEE THE COURSE WEBSITE FOR DOCUMENTATION AND INFORMATION

NAME:

SURNAME:

ID:

SIGNATURE:

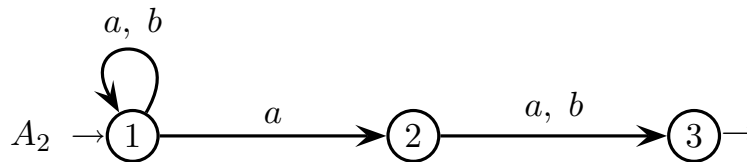
INSTRUCTIONS - READ CAREFULLY:

- The exam consists of two parts:
 - I (80%) Theory:
 1. regular expressions and finite automata
 2. free grammars and pushdown automata
 3. syntax analysis and parsing
 4. translation and semantic analysis
 - II (20%) Practice on Flex and Bison
- To pass the exam, the candidate must succeed in both parts (I and II), in one call or more calls separately, but within one year.
- To pass part I (theory) one must answer the mandatory (not optional) questions. Notice the full grade is achieved by answering the optional questions as well.
- The exam is open book (texts and personal notes are admitted).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets nor replace the existing ones.
- Time: Part I (theory): 2h.15m - Part II (practice): 60m

1 Regular Expressions and Finite Automata 20%

1. Consider the regular expression R and the finite state automaton A_2 below:

$$R = b^* (a b^+)^*$$



Answer the following questions:

- (a) Take the regular expression R given above:
- In alphabetical order, list the strings of length exactly four that belong to the regular language $L(R)$, and informally describe (that is describe by words) such a language.
 - By means of the Berry-Sethi algorithm, obtain a deterministic finite state automaton A_1 that recognises the regular language $L(R)$; and if it is necessary, minimise automaton A_1 .
 - (optional) Say if the regular language $L(R)$ is local and explain why.
- (b) Take the finite state automaton A_2 given above:
- Say which strings given in response to point (a).i are accepted by automaton A_2 as well.
 - Build the product automaton $A_3 = A_1 \times A_2$ that recognises language $L(R) \cap L(A_2)$; say if the product automaton A_3 is deterministic or not, and explain why.
 - Write the execution traces of the product automaton A_3 that recognise the strings listed at point (b).i.
-

2 Free Grammars and Pushdown Automata 20%

1. Consider an alphabet $\Sigma = \{0, 1, '(', ')'\}$; a *binary* $y \in \{0, 1\}^+$ is a non-empty string of bits. A language L_1 is recursively defined by the three following conditions. Let strings x , x_1 and x_2 be arbitrary phrases belonging to language L_1 ; here are the recursive conditions:

- every binary y belongs to L_1
- every string “(x)” belongs to L_1
- every concatenation “ $x_1 x_2$ ” belongs to L_1

Examples and counterexamples:

<i>valid</i>	00110	
<i>valid</i>	(0 (10) (110) 100 (10))	
<i>valid</i>	(1100) 0 ((10)) (11 (110) 0) 100	
<i>not valid</i>	(0 () (110) 100 (10))	- empty bracket pair
<i>not valid</i>	(0 (11) (110)	- unbalanced bracket

Answer the following questions:

- (a) Write a grammar G_1 , not in the extended form (BNF), that generates language L_1 , and check that grammar G_1 is not ambiguous.
- (b) In a phrase of language L_1 , consider a pair of corresponding brackets. The *binary number* associated with them, is the sequence of bits contained in such brackets, but not contained in any more internal brackets. The bits that are not included in any bracket pair, constitute a binary number as well. See the sample strings and binary numbers in the table below.

Write a grammar G_2 , not in the extended form (BNF), that generates the language $L_2 \subset L_1$ defined in this way:

$$x \in L_2 \text{ if and only if } \begin{cases} x \in L_1 & \text{and} \\ \text{every binary number in } x \text{ ends with a bit } 0 \end{cases}$$

Examples, where the barred binary numbers are those that invalidate the string:

		<i>contained binary numbers</i>
<i>valid</i>	0110 (10)	0110, 10
<i>valid</i>	0 (10) 100 (0 (0) 0)	0100, 10, 00, 0
<i>not valid</i>	(0 (11) (0) (10) 1)	01 , 11 , 0, 10
<i>not valid</i>	11	11

2. One wishes one grammatically modeled the construct **case** of the historic programming language Pascal, which realises a multiple way conditional. Here is an example:

```
case 2 * (x - y) of

    (* branch taken if expression 2 * (x - y) equals 1      *)
    1 : begin
        a := b + ( c - 1 ) * d;
        b := c - 1
    end;

    (* branch taken if expression 2 * (x - y) equals 3 or 4 *)
    3, 4 : a := 2;

    5 : ;          (* a branch may be empty *)

    -8 : begin     (* blocks may be nested  *)
        ...
    end;

    (* branch taken in all the other cases                    *)
    otherwise ...

end
```

The comments between **(*** and ***)** do not belong to the language. The specifications below hold:

- identifiers and constants are modeled by terminals **id** and **num**, respectively
- expressions are numerical, with variables and constants, operators of addition '+', subtraction '-', and multiplication '*', and round brackets '(' e ')'; the operators precedences are as usual (operator '*' precedes operators '+' and '-')
- assignments ':=' have a variable on the left and an expression on the right
- the labels of the branches of **case**, constitute a non-empty list of numerical constants, with a separator comma ',' and a terminator colon ':'
- the clause **otherwise** is optional and may show only at the end
- there must be at least one branch in a **case**, different from **otherwise**
- a semicolon ';' is optional before the keyword **end** in a block **begin-end**
- the construct **case** may be nested (recursively)

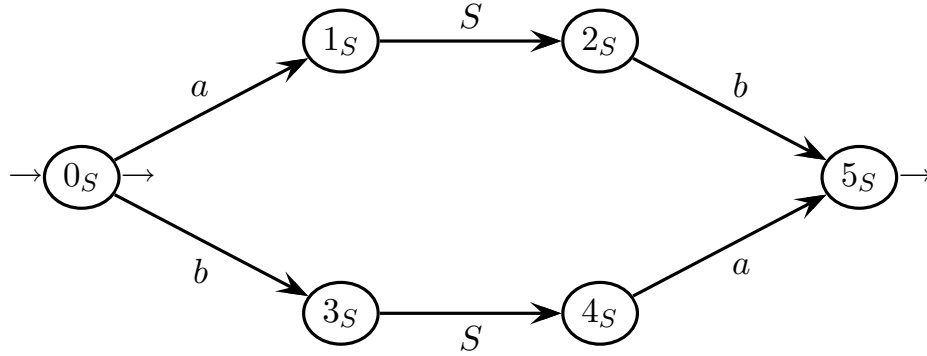
If it is necessary, imagine to reasonably complete the construct **case** described before, and explain those choices of yours that are not prescribed by the example or the specifications above.

Write a grammar in the extended form (EBNF), not ambiguous, that models the construct **case** of Pascal sketched above.

3 Syntax Analysis and Parsing 20%

1. Consider the grammar G below (axiom S), represented by means of its machine network, which here consists of only one recursive machine:

$$G: S \rightarrow a S b \mid b S a \mid \varepsilon$$



Answer the following questions (and use the machine network representation):

- (a) Show that grammar G is not of type $ELR(1)$, by building the smallest possible portion of the pilot graph of G .
- (b) Use the simplified direct construction algorithm for the control flow graph $PCFG$ of grammar G , and show on the $PCFG$ that grammar G is not of type $ELL(1)$. Explain in detail how to compute the most significant guide sets.
- (c) Apply the Early algorithm to grammar G and analyse string $abab$. With reference to the contents of the Earley vector E , show that the string is accepted, as well as it is accepted its shorter prefix string ab , while its prefix string aba is rejected. To apply the Earley algorithm, use the table prepared on the next page (the number of rows is not significant).
- (d) (optional) Say if the language $L(G)$ generated by grammar G is deterministic or not, and explain why.

FOR WHAT FOLLOWS SEE THE WARNING ON THE TEXT COVER PAGE

Those who prefer to answer by means of the classical LR and LL analysis methodologies, please use grammar G , which is already not in the extended form (BNF), instead of the machine network.

[illegible]

4 Translation and Semantic Analysis 20%

1. Every phrase of the source language L_s has the (regular) structure $(a^+ b^+)^+$. Therefore a generic phrase of L_s is structured as a series of blocks, this way:

$$a^{m_1} b^{n_1} \dots \overbrace{a^{m_i} b^{n_i}}^{block} \dots a^{m_k} b^{n_k} \quad \text{with } k \geq 1$$

Consider the transduction τ such that every *block* is transformed as follows:

$$a^{m_i} b^{n_i} \xrightarrow{\tau} \begin{cases} c^{m_i+n_i} & \text{if } m_i \geq n_i \\ d^{n_i-m_i} & \text{if } m_i < n_i \end{cases}$$

Example:

$$a^2 b^2 a^2 b^3 a^3 b \xrightarrow{\tau} c^4 d c^4$$

Answer the following questions:

- (a) Write a syntax transduction scheme (G_s, G_d) or equivalently a transduction grammar G_τ , that defines transduction τ ; draw the source and destination syntax trees separately, or unify them, for the sample string $a^2 b^2 a^2 b^3 a^3 b$.
- (b) (optional) Say if transduction τ is computable by a deterministic pushdown transducer A_τ , and explain why.

2. One wishes one translated an expression from the infix to the postfix form, through an attribute grammar instead of a syntax transduction. Here is a syntax support for the expression to translate, which is unambiguous and right associative (axiom S):

$$\left\{ \begin{array}{ll} S \rightarrow E & \\ E \rightarrow T + E & T \rightarrow (E) \\ E \rightarrow T & T \rightarrow a \end{array} \right.$$

The expression model is quite simple: it only has the addition operator and brackets; terminal a generically represents variables and constants.

In particular two different transductions are envisioned, called ρ and σ , this way:

- Transduction ρ computes the usual postfix form (by associating on the right) and of course it *eliminates* the brackets. Example:

a source string $a +_1 a +_2 (a +_3 a) +_4 a$

is translated into $\rho = a a a a +_3 a +_4 +_2 +_1$

- Transduction σ computes the usual postfix form (by associating on the right), but only inside of an *odd* number of nested brackets, while elsewhere (i.e., inside of an *even* number of nested brackets) it leaves the infix form unchanged, and of course it *preserves* the brackets. The parts of the expression that are outside of any bracket pair, are considered to be at 0 (i.e., even) nesting level. Example:

the source string $a +_1 (a +_2 (a +_3 a) +_4 a)$

is translated into $\sigma = a +_1 (a (a +_3 a) a +_4 +_2)$

In the two examples above, numbering the operator ‘+’ helps us visually understand how the translation works, but it does not belong to the grammar.

In both cases the translated expression must be represented by means of an attribute τ of type string, which is emitted complete at the tree root. The tables to be filled with the attributes and the semantic functions, are prepared on the next pages.

Answer the following questions:

- Write an attribute grammar G_ρ that computes the transduction ρ .
- Write an attribute grammar G_σ that computes the transduction σ , and explain if grammar G_σ is of type one-sweep.
- (optional) Decorate the syntax tree of the sample string shown above for transduction σ , with the attributes and their values, and also with the functional dependences among the attributes. The syntax tree to be decorated is prepared on the last page.

attributes to be used for the grammars - questions (a) and (b)

type	name	(non)terminals	domain	meaning
left	τ	S, E, T	string	translated string (according to ρ or σ)

syntax	attribute computation for transduction ρ - question (a)
$S_0 \rightarrow E_1$	
$E_0 \rightarrow T_1 + E_2$	
$E_0 \rightarrow T_1$	
$T_0 \rightarrow (E_1)$	
$T_0 \rightarrow a$	

syntax	attribute computation for transduction σ - question (b)
$S_0 \rightarrow E_1$	
$E_0 \rightarrow T_1 + E_2$	
$E_0 \rightarrow T_1$	
$T_0 \rightarrow (E_1)$	
$T_0 \rightarrow a$	

syntax tree of string $a + (a + (a + a) + a)$ to be decorated

